МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №8

по дисциплине «Вычислительная математика»

Тема: Решение систем линейных уравнений методом Якоби.

Студент гр. 4342 Закаблук А.Ю.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Реализовать метод Якоби, протестировать его на матрицах различного размера (включая матрицы большого размера), изучить условия сходимости итерационных методов и сравнить точность с существующими реализациями методов решения СЛУ.

Теоретический минимум.

Метод Якоби (метод простых итераций) — это итерационный алгоритм решения систем линейных уравнений $Ax=b$. В отличие от прямых методов (таких как метод Гаусса), он не находит точное решение за конечное число шагов, а строит бесконечную последовательность векторов $x^{(0)}$, $x^{(1)}$, …, которая сходится к точному решению.

Исходная система преобразуется к эквивалентному итерационному виду:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Достаточное условие сходимости**:** Метод Якоби гарантированно сходится при любом начальном приближении $x^{(0)}$, если матрица A обладает **строгим диагональным преобладанием** (модуль диагонального элемента строго больше суммы модулей остальных элементов в строке).

Вычисления прекращаются, когда максимальная разница между текущим и предыдущим приближением становится меньше заданной точности: $\max|x_i^{(k+1)} - x_i^{(k)}| \leq \varepsilon$.

Для оценки итоговой погрешности используется Евклидова норма (L2-норма) вектора ошибки: $e=||\bar{x} - x*||_2$.

Временная сложность и эффективность метода: Вычислительная (временная) сложность метода Якоби составляет $O(K \cdot N^2)$, где N — размерность матрицы, а K — количество итераций, необходимых для достижения точности ε.

Обоснование сложности: На каждой отдельной итерации для вычисления одного нового значения x_i требуется выполнить сумму из $N-1$ произведений.

Так как вектор состоит из N переменных, один полный шаг обновления всего вектора x требует выполнения ровно $N\cdot(N-1)\approx N^2$ арифметических операций. Для получения итогового ответа этот шаг повторяется K раз.

Преимущества перед прямыми методами: Временная сложность метода Гаусса составляет $O(N^3)$. Если матрица имеет сильное диагональное преобладание, алгоритм Якоби сходится очень быстро, и число итераций K оказывается многократно меньше размера матрицы ($K \ll N$). В таком случае итерационный алгоритм работает на порядки быстрее метода Гаусса. Кроме того, метод Якоби не изменяет исходную матрицу A в процессе вычислений (отсутствует «проблема заполнения» нулей ненулевыми элементами), что делает его идеальным выбором для решения сверхбольших разреженных и трехдиагональных матриц.

Выполнение работы.

В качестве метода решения был реализован метод Якоби. При запуске программы создается квадратная матрица A размера $N\times N$, заполняемая случайными числами. Так как метод Якоби расходится для случайных матриц, к сгенерированной матрице принудительно **применяется преобразование для обеспечения диагонального преобладания** (к каждому диагональному элементу прибавляется сумма модулей остальных элементов строки).

Затем генерируется эталонный вектор решений $\bar{x}$, вычисляется вектор свободных членов b, и данные передаются в функцию JacobiMethod. Алгоритм итеративно вычисляет приближение до тех пор, пока разница между шагами не станет меньше $\varepsilon=10^{-8}$. Предусмотрена защита от зацикливания: если число итераций превысит 10000, алгоритм прервется с ошибкой расходимости.

Тестирование алгоритма:

1. **Сравнение с библиотекой Eigen:**

Проведено сравнение скорости и точности нашей реализации метода Якоби с прямым методом (partialPivLu) из библиотеки Eigen. Для диагонально-

преобладающих матриц метод Якоби показывает великолепную скорость сходимости, сопоставимую или даже превосходящую прямые методы на больших размерностях.

2. **Стресс-тестирование (Матрицы большого размера):**

Запуск алгоритма на матрицах N=1000 и N=10000 прошел успешно. Итерационные методы требуют меньше памяти, так как не меняют исходную матрицу, что делает их идеальными для больших разреженных систем.

```
--- Тестирование на большой матрице N = 1000 ---
Матрица готова. Запуск итераций Якоби...
УСПЕХ! Время решения: 0.0592916 секунд

--- Тестирование на большой матрице N = 10000 ---
Матрица готова. Запуск итераций Якоби...
УСПЕХ! Время решения: 4.53945 секунд
```

3. **Спец. матрицы (Матрицы Гильберта):**

Алгоритм был запущен на плохо обусловленной матрице Гильберта (без принудительного изменения диагонали). Как и ожидалось из теории, матрица Гильберта не обладает диагональным преобладанием, поэтому **метод Якоби мгновенно расходится**, превышая лимит итераций. Это наглядно демонстрирует главное ограничение метода: он применим только к строго определенному классу матриц..

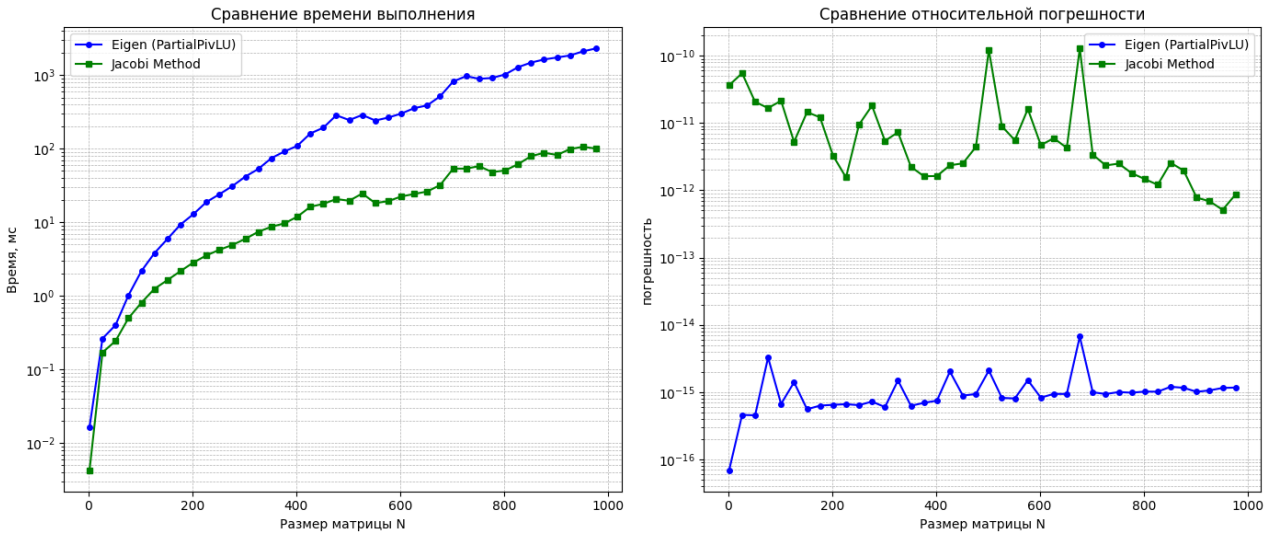

Рисунок 1 - Сравнение времени выполнения и относительной погрешности

Выводы.

Был успешно реализован и протестирован итерационный метод Якоби. Подтверждено теоретическое условие сходимости: алгоритм быстро и с высочайшей точностью находит решение для матриц со строгим диагональным преобладанием, справляясь даже с огромными размерностями (10000×10000). Однако тестирование на матрице Гильберта показало уязвимость алгоритма: отсутствие диагонального преобладания приводит к полному отсутствию сходимости. В отличие от универсального метода Гаусса, метод Якоби требует предварительного анализа свойств системы перед применением.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <vector>
#include <chrono>
#include <stdexcept>
#include <windows.h>

#include "Eigen/Dense"
#include "matrixlib.hpp"

#define REPEAT_COUNT 10

using type = double;

template <class T>
std::pair<std::vector<T>, std::vector<T>>
getCorrectResponseVectors(squareMatrix<T>& M) {
    int N = M.getN();
    std::vector<T> X(N, 0), B(N, 0);
    for (int i = 0; i < N; ++i) {
        X[i] = M.getRandomNumber();
    }
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            B[i] += (M.getElement(i, j) * X[j]);
        }
    }
    return {X, B};
}

// РЕАЛИЗАЦИЯ МЕТОДА ЯКОБИ
template <class T>
std::vector<T> JacobiMethod(squareMatrix<T>& M, const std::vector<T>& B) {
    int N = M.getN();
    std::vector<T> X(N, 0.0); // Начальное приближение: нули
    std::vector<T> X_new(N, 0.0);
    T diff;
    int iterations = 0;
    const int MAX_ITER = 10000; // Защита от бесконечного цикла

    // Проверка диагонали на нули
    for (int i = 0; i < N; ++i) {
        if (std::fabs(M.getElement(i, i)) < EPSILON) {
            throw std::runtime_error("Нулевой элемент на главной диагонали! Метод
неприменим.");
        }
    }

    do {
        diff = 0.0;
        for (int i = 0; i < N; ++i) {
            T sum = B[i];
            for (int j = 0; j < N; ++j) {
                if (i != j) {
                    sum -= M.getElement(i, j) * X[j];
                }
            }
```

```
            X_new[i] = sum / M.getElement(i, i);

            // Ищем максимальную разность по модулю (норма бесконечности)
            if (std::fabs(X_new[i] - X[i]) > diff) {
                diff = std::fabs(X_new[i] - X[i]);
            }
        }

        X = X_new;
        iterations++;

        if (iterations >= MAX_ITER) {
            throw std::runtime_error("Метод Якоби расходится (превышен лимит итераций). Нет диагонального преобладания.");
        }
    } while (diff > EPSILON);

    return X;
}

int getData() {
    std::fstream log("log.txt", std::ios::out);
    std::vector<std::vector<long double>> results;
    results.reserve(200);

    std::cout << "Начинаю сбор статистики (от N=2 до N=2000). Это займет некоторое время...\n";

    for (int i = 2; i <= 2000; i += 20) {
        int count = 0;
        std::vector<long double> row(5, 0.0);
        row[0] = i;
        while(count++ != REPEAT_COUNT) {
            squareMatrix<type> M_original(i);
            M_original.makeDiagonallyDominant();  // ВАЖНО: Делаем матрицу решаемой для Якоби

            auto vectors = getCorrectResponseVectors<type>(M_original);
            squareMatrix<type> M_for_jacobi = M_original;

            Eigen::MatrixXd eigenM(i, i);
            Eigen::VectorXd eigenB(i);
            for (int k = 0; k < i; ++k) {
                for (int j = 0; j < i; ++j) {
                    eigenM(k, j) = M_original.getElement(k, j);
                }
                eigenB(k) = vectors.second[k];
            }

            auto startEigen = std::chrono::high_resolution_clock::now();
            Eigen::VectorXd eigenX = eigenM.partialPivLu().solve(eigenB);
            auto endEigen = std::chrono::high_resolution_clock::now();
            row[1] += std::chrono::duration<double, std::milli>(endEigen - startEigen).count();

            long double err_eigen = 0.0;
            int valid_count_eigen = 0;
            for (int k = 0; k < i; ++k) {
                if (std::fabs(vectors.first[k]) > 1e-10) {
                    double d = std::fabs(vectors.first[k] - eigenX(k));
                    err_eigen += d / std::fabs(vectors.first[k]);
                    valid_count_eigen++;
                }
```

```
            }
            row[2] += (valid_count_eigen > 0) ? err_eigen / valid_count_eigen :
0.0;

            auto start = std::chrono::high_resolution_clock::now();
            try {
                std::vector<type>   answer   =   JacobiMethod<type>(M_for_jacobi,
vectors.second);
                auto end = std::chrono::high_resolution_clock::now();
                row[3]   +=   std::chrono::duration<double,   std::milli>(end   -
start).count();

                long double err_jacobi = 0.0;
                int valid_count_jacobi = 0;
                for (int k = 0; k < i; ++k) {
                    if (std::fabs(vectors.first[k]) > 1e-10) {
                        double d = std::fabs(vectors.first[k] - answer[k]);
                        err_jacobi += d / std::fabs(vectors.first[k]);
                        valid_count_jacobi++;
                    }
                }
                row[4]   +=   (valid_count_jacobi   >   0)   ?   err_jacobi   /
valid_count_jacobi : 0.0;
            } catch (...) {
                count--;
                continue;
            }
        }
        row[1] /= REPEAT_COUNT;
        row[2] /= REPEAT_COUNT;
        row[3] /= REPEAT_COUNT;
        row[4] /= REPEAT_COUNT;
        results.push_back(row);

        if (i % 200 == 0 || i == 2) std::cout << "Обработано N = " << i << "...\n";
    }

    for (const auto& row : results) {
        for (auto val : row) log << val << '\t';
        log << '\n';
    }
    log.close();
    std::cout << "Данные сохранены в log.txt. Генерирую графики...\n";

    // Python скрипт (исправлены подписи на Якоби)
    std::fstream pyScript("plotter.py", std::ios::out);
    pyScript << "import matplotlib.pyplot as plt\nimport sys\n\n";
    pyScript << "N, t_eigen, err_eigen, t_jacobi, err_jacobi = [], [], [], [],
[]\n";
    pyScript << "try:\n    with open('log.txt', 'r') as f:\n        for line in
f:\n            data = list(map(float, line.split()))\n";
    pyScript << "          N.append(data[0])\n          t_eigen.append(data[1])\n
err_eigen.append(data[2])\n";
    pyScript   <<   "                                t_jacobi.append(data[3])\n
err_jacobi.append(data[4])\n";
    pyScript  <<  "except  Exception  as  e:\n      print('Ошибка  чтения:',  e)\n
sys.exit(1)\n\n";
    pyScript << "fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))\n\n";

    pyScript   <<   "ax1.plot(N,   t_eigen,   'b-o',   markersize=4,   label='Eigen
(PartialPivLU)')\n";
    pyScript   <<   "ax1.plot(N,   t_jacobi,   'g-s',   markersize=4,   label='Jacobi
Method')\n";
```

```
    pyScript << "ax1.set_yscale('log')\nax1.grid(True, which='both', linestyle='-
-', linewidth=0.5)\n";
    pyScript << "ax1.set_title('Сравнение времени
выполнения')\nax1.set_xlabel('Размер матрицы N')\nax1.set_ylabel('Время,
мс')\nax1.legend()\n\n";

    pyScript << "ax2.plot(N, err_eigen, 'b-o', markersize=4, label='Eigen
(PartialPivLU)')\n";
    pyScript << "ax2.plot(N, err_jacobi, 'g-s', markersize=4, label='Jacobi
Method')\n";
    pyScript << "ax2.set_yscale('log')\nax2.grid(True, which='both', linestyle='-
-', linewidth=0.5)\n";
    pyScript << "ax2.set_title('Сравнение относительной
погрешности')\nax2.set_xlabel('Размер матрицы
N')\nax2.set_ylabel('погрешность')\nax2.legend()\n\n";

    pyScript << "plt.tight_layout()\nplt.savefig('graphs_jacobi.png',
dpi=300)\nplt.show()\n";
    pyScript.close();
    system("python plotter.py");

    return EXIT_SUCCESS;
}

int CompareJacobiAndLU() {
    int N = 0;
    std::cout << "Введите размер матрицы N: ";
    std::cin >> N;
    squareMatrix<type> M(N);
    M.makeDiagonallyDominant(); // Гарантируем сходимость
    auto vectors = getCorrectResponseVectors<type>(M);

    Eigen::MatrixXd eigenM(N, N);
    Eigen::VectorXd eigenB(N);
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            eigenM(i, j) = M.getElement(i, j);
        }
        eigenB(i) = vectors.second[i];
    }

    auto startEigen = std::chrono::high_resolution_clock::now();
    Eigen::VectorXd eigenX = eigenM.partialPivLu().solve(eigenB);
    auto endEigen = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> durationEigen = endEigen -
startEigen;
    std::cout << "Время решения Eigen (LU): " << durationEigen.count() << " мс"
<< std::endl;

    try {
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<type> answer = JacobiMethod<type>(M, vectors.second);
        auto end = std::chrono::high_resolution_clock::now();

        std::chrono::duration<double, std::milli> duration = end - start;
        std::cout << "Время метода Якоби: " << duration.count() << " мс" <<
std::endl;

        long double diffNorm = 0.0;
        long double errorNorm = 0.0;
        for (int i = 0; i < N; ++i) {
            diffNorm += (vectors.first[i] - eigenX(i))*(vectors.first[i] -
eigenX(i));
```

```
            errorNorm  +=  (vectors.first[i]  -  answer[i])*(vectors.first[i]  -
answer[i]);
        }
        std::cout  <<  "||Точный_X  -  Eigen_X||_2  =  "  <<  std::sqrt(diffNorm)  <<
std::endl;
        std::cout << "||Точный_X - Jacobi_X||_2 = " << std::sqrt(errorNorm) <<
std::endl;
    } catch (const std::exception& e) {
        std::cout << "Ошибка метода Якоби: " << e.what() << std::endl;
    }
    return EXIT_SUCCESS;
}

int TestLargeMatrices() {
    std::vector<int> sizes = {1000, 10000};
    for (int N : sizes) {
        std::cout << "\n--- Тестирование на большой матрице N = " << N << " ---
\n";
        try {
            squareMatrix<type> M(N);
            M.makeDiagonallyDominant();
            auto vectors = getCorrectResponseVectors<type>(M);

            std::cout << "Матрица готова. Запуск итераций Якоби..." << std::endl;
            auto start = std::chrono::high_resolution_clock::now();
            std::vector<type> answer = JacobiMethod<type>(M, vectors.second);
            auto end = std::chrono::high_resolution_clock::now();

            std::chrono::duration<double> duration = end - start;
            std::cout << "УСПЕХ! Время решения: " << duration.count() << " секунд"
<< std::endl;
        } catch (const std::bad_alloc& e) {
            std::cout << "Ошибка: Недостаточно ОЗУ для матрицы " << N << "x" << N
<< std::endl;
        } catch (const std::exception& e) {
            std::cout << "Ошибка: " << e.what() << std::endl;
        }
    }
    return EXIT_SUCCESS;
}

int TestHilbertMatrix() {
    int N = 0;
    std::cout << "Введите размер матрицы Гильберта N: ";
    std::cin >> N;

    squareMatrix<type> M(N, true);
    // ВНИМАНИЕ: Мы НЕ вызываем makeDiagonallyDominant(), так как хотим показать,
что Якоби упадет!
    auto vectors = getCorrectResponseVectors<type>(M);

    try {
        std::cout << "Запуск метода Якоби на матрице Гильберта...\n";
        std::vector<type> answer = JacobiMethod<type>(M, vectors.second);
        std::cout << "Решено успешно!" << std::endl;
    } catch (const std::exception& e) {
        std::cout << "\n[ОЖИДАЕМАЯ ОШИБКА]: " << e.what() << std::endl;
        std::cout << "Матрица Гильберта не обладает диагональным преобладанием,
метод Якоби не применим.\n";
    }
    return EXIT_SUCCESS;
}
```

```
int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    std::cout << "МЕНЮ РЕШЕНИЯ СЛУ МЕТОДОМ ЯКОБИ:\n";
    std::cout << "1 - Получить данные для построения графиков сравнения\n";
    std::cout << "2 - Сравнить метод Якоби с библиотекой Eigen (N на N)\n";
    std::cout << "3 - СТРЕСС-ТЕСТ: матрицы большого размера (1000 и 10000)\n";
    std::cout << "4 - СПЕЦ. МАТРИЦЫ: Тестирование на матрице Гильберта (Тест на
расходимость)\n";

    int choice;
    std::cin >> choice;
    switch (choice)
    {
    case 1: return getData();
    case 2: return CompareJacobiAndLU();
    case 3: return TestLargeMatrices();
    case 4: return TestHilbertMatrix();
    default: return 0;
    }
}
```

Файл matrixlib.hpp

```
#ifndef MATRIXLIB_HPP
#define MATRIXLIB_HPP

#include <limits>
#include <string>
#include <vector>
#include <iostream>
#include <fstream>
#include <iomanip>
#include <random>
#include <type_traits>

#define STANDART_MATRIX_SIZE 5
#define MAXIMUM_ELEMENT_VALUE 10
#ifndef EPSILON
#define EPSILON 1e-9
#endif

template <class T>
class squareMatrix {
private:
    std::vector<std::vector<T>> M;
    int N;
    bool symmetric;
    bool singular;
public:
```

```
    squareMatrix(int matrixSide, bool isHilbert = false);
    squareMatrix();

    int getN() const { return this->N; }
    const T& getElement(int rowIndex, int columnIndex) const { return this-
>M[rowIndex][columnIndex]; }
    void setElement(int rowIndex, int columnIndex, T value) { this-
>M[rowIndex][columnIndex] = value; }

    T getRandomNumber();
    T getGilbertRandom(int i, int j);

    int getRowOfMaxElement(int colomn, int startRow) const;
    void swapRows(int rowI, int rowJ);
    std::vector<T>& getRow(int index) { return this->M[index]; }
    void makeDiagonallyDominant() {
        for (int i = 0; i < N; ++i) {
            T sum = 0;
            for (int j = 0; j < N; ++j) {
                if (i != j) {
                    sum += std::fabs(M[i][j]);
                }
            }
            // Гарантируем строгое диагональное преобладание
            M[i][i] = sum + static_cast<T>(10.0);
        }
    }
};

template <class T>
squareMatrix<T>::squareMatrix(int matrixSide, bool isHilbert) : N(matrixSide) {
    M.resize(N, std::vector<T>(N));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (isHilbert) {
                M[i][j] = getGilbertRandom(i, j);
            } else {
                M[i][j] = getRandomNumber();
            }
        }
    }
    this->symmetric = false;
```

```
    this->singular = false;
}

template <class T>
squareMatrix<T>::squareMatrix() : squareMatrix(STANDART_MATRIX_SIZE, false) {}

template <class T>
T squareMatrix<T>::getGilbertRandom(int i, int j) {
    // Формула элемента матрицы Гильберта H(i,j) = 1 / (i + j + 1)
    return static_cast<T>(1.0 / (i + j + 1));
}

template <class T>
T squareMatrix<T>::getRandomNumber() {
    if constexpr (std::is_same_v<T, int>) {
        static std::random_device rd;
        static std::mt19937 gen(rd());
        static std::uniform_int_distribution<int> dist(-MAXIMUM_ELEMENT_VALUE,
MAXIMUM_ELEMENT_VALUE);
        return dist(gen);
    }
    else if constexpr (std::is_same_v<T, double> || std::is_same_v<T, float> ||
std::is_same_v<T, long double>) {
        static std::random_device rd;
        static std::mt19937 gen(rd());
        static std::uniform_real_distribution<long double> dist(-
MAXIMUM_ELEMENT_VALUE, MAXIMUM_ELEMENT_VALUE);
        return static_cast<T>(dist(gen));
    }
    return static_cast<T>(0);
}

template <class T>
int squareMatrix<T>::getRowOfMaxElement(int colomn, int startRow) const {
    int targetRow = -1;
    T maxElem = -1.0;
    for(int i = startRow; i < N; ++i) {
        const T& currentElement = std::fabs(M[i][colomn]);
        if (currentElement > maxElem) {
            targetRow = i;
            maxElem = currentElement;
        }
```

```
    }
    return targetRow;
}

template <class T>
void squareMatrix<T>::swapRows(int rowI, int rowJ) {
    if(rowI < N && rowJ < N && rowI >= 0 && rowJ >= 0) {
        std::swap(M[rowI], M[rowJ]);
    }
}

#endif // MATRIXLIB_HPP
```